TECHNICAL DEEP-DIVE · QUERY RE-ENGINEERING

From Monolith to Native HANA

A side-by-side walkthrough of how the production-planning query was transformed, statement by statement, from SQL Server T-SQL to SAP HANA SQLScript. Companion to the PACE method report.

T-SQL → SQLScript | Confidential · sanitized excerpts

Author **Raúl** Focus **Query transformations** Comparisons **7 patterns** Behavior **Preserved 1:1**

DATA INTEGRITY

To protect the client's data model, every excerpt below is **simplified and sanitized**: table names, custom field names, item-code conventions and business constants have been replaced with generic placeholders. The **transformation patterns are faithful** to the real implementation — only the identifying values are masked, so the engineering is fully visible while the underlying data stays private.

READING THE EXCERPTS — MASKED NAMES

- `SALES_ORD / SALES_LN` sales order header / lines
- `PROD_ORD / PROD_LN` production order header / lines
- `ITEM` item master
- `ACT_LOG / ACT_TYPE` activity log / activity types
- `@CLONE · @STAGE · @RATE` configuration tables
- `<family> · :MIN_FLOOR · <window>` masked business values

01 Query transformations

Seven representative changes, each showing the legacy statement, the rebuilt statement, and the reason.

01 Backup factors: IIF pyramids → documented constant

CONDITIONAL LOGIC

Why: safety-stock backups were computed with IIF nested inside scalar subqueries — unauditable, HANA-incompatible, and effectively just a floor value.

BEFORE · T-SQL

```
@RESLAB FLOAT = (
 SELECT IIF(
   ( demand / supply ) > 2,
     demand / supply,    -- ratio
     2 ))                -- else floor
```

AFTER · SQLSCRIPT

```
DECLARE v_resp_lab
   DECIMAL(5,2) := 2.00;
-- 200% floor, self-documenting;
-- a live ratio (where still
-- needed) is NULLIF-guarded.
```

WHAT CHANGED An IIF-over-subquery pyramid collapses to one explicit, named parameter; the dead complexity is gone.

02 Date windows: DATEADD / GETDATE → ADD_DAYS / CURRENT_DATE

DATE FUNCTIONS

Why: T-SQL date functions don't exist in HANA, and week-based offsets have no direct equivalent.

BEFORE · T-SQL

```
WHERE o.DueDate BETWEEN
   DATEADD(WW,-52,GETDATE())
   AND DATEADD(WW,-40,GETDATE())
-- and DATEPART(ww, DueDate)
```

AFTER · SQLSCRIPT

```
WHERE o."DueDate" BETWEEN
   ADD_DAYS(CURRENT_DATE,-364)
   AND ADD_DAYS(CURRENT_DATE,-280)
-- and WEEK("DueDate")
```

WHAT CHANGED Week offsets rewritten as day offsets; DATEPART → WEEK / YEAR / MONTH; identifiers quoted for HANA's case-sensitivity.

03 Scalar variables: DECLARE @x = (SELECT…) → SELECT … INTO

VARIABLE ASSIGNMENT

Why: SQLScript can't assign a variable from an inline subquery, and an empty window must not yield NULL.

BEFORE · T-SQL

```
DECLARE @wk6 INT = (
 SELECT SUM(l.Qty)/6
 FROM SALES_LN l
 JOIN SALES_ORD h ON …
 WHERE … )
```

AFTER · SQLSCRIPT

```
SELECT IFNULL(SUM(l."Qty")/6, 0)
INTO v_wk6
FROM SALES_LN l
JOIN SALES_ORD h ON …
WHERE … ;
```

WHAT CHANGED Inline assignment becomes `SELECT … INTO`; `IFNULL(…, 0)` makes an empty result a real zero instead of `NULL`.

04 Temp tables & strict typing: #tmp → LOCAL TEMPORARY TABLE

STORAGE & TYPES

Why: HANA uses different temp-table syntax and will not silently coerce text to numbers or mismatched join keys.

BEFORE · T-SQL

```
CREATE TABLE #CALIDAD (
  Producto VARCHAR(12),
  Calidad FLOAT)
-- def_1 + def_2 + def_3 …
-- ON a.DocNum = b.DocNum
```

AFTER · SQLSCRIPT

```
CREATE LOCAL TEMPORARY TABLE
  #TEMP_CALIDAD (
  "Producto" NVARCHAR(20),
  "Calidad" DECIMAL(10,2))
-- IFNULL(TO_DECIMAL(def_1,10,2),0)…
-- ON a=b via TO_NVARCHAR(b.DocNum)
```

WHAT CHANGED `#tmp` → `LOCAL TEMPORARY TABLE`; `FLOAT` → typed `DECIMAL`; every field cast with `TO_DECIMAL / TO_INT / TO_NVARCHAR`, including type-safe join keys.

05 Division safety across the stage chain

NULL / ZERO HANDLING

Why: each stage divides by the previous stage's duration and factor — any zero or missing value broke the whole backward chain.

BEFORE · T-SQL

```
@INV FLOAT = (
 SELECT
   ((@VIV / weeks) * dur)
      / factor
 FROM #Factores
 WHERE Stage = 8)
```

AFTER · SQLSCRIPT

```
SELECT IFNULL(
  ((:v_VIV / NULLIF(F9."Dur",0))
     * F8."Dur")
  / NULLIF(COALESCE(F8."Fac",
            F8."FacRate"),0), 0)
INTO v_INV FROM #Factores F8 …
WHERE F8.Stage = '08';
```

WHAT CHANGED Every denominator wrapped in `NULLIF(x,0)`, a missing factor recovered via `COALESCE`, and the whole result `IFNULL(…,0)` — the chain can no longer fail.

06 Killing the duplication: 9× UNION ALL → one COALESCE

BIGGEST WIN

Why: the rule "use this clone's factor, else fall back to the default clone" was copy-pasted as nine near-identical blocks, one per stage (~15 lines each).

BEFORE · T-SQL — × 9 BLOCKS

```
-- repeated for stages 01…09
SELECT …,
 CASE WHEN (SELECT Fac … Stage='0X'
        AND Clone=@CLON) IS NULL
   THEN (SELECT Fac … Stage='0X'
        AND Clone='01') END
FROM #P2 FULL JOIN #Base …
WHERE Stage='0X'
UNION ALL …  (×9)
```

AFTER · SQLSCRIPT — ONCE, ALL STAGES

```
COALESCE(
  clone_factor,   -- this clone
  default_factor  -- fallback
) AS "FactorFinal"

-- evaluated per row, for every
-- stage at once. ~135 lines → 1.
```

WHAT CHANGED Roughly **135 lines of copy-paste collapse into a single COALESCE** applied uniformly to all stages — the same behavior, a fraction of the surface area.

07 Hardcoded windows → data-driven durations & parameters

CONFIGURATION OVER CODE

Why: each product family had its stage length hardcoded as day ranges scattered through the query; retuning a stage meant editing the procedure.

BEFORE · T-SQL

```
WHERE Product LIKE '%<fam>'
  AND Date BETWEEN
    DATEADD(DD,-<window>,GETDATE())
    AND DATEADD(DD,-28,GETDATE())
OR Product LIKE '%<fam2>' …
-- one branch per family
```

AFTER · SQLSCRIPT

```
LEFT JOIN "@STAGE_DURATION" d
  ON d."Product"=i."ItemCode"
 AND d."Month"=MONTH(a."Date")
WHERE a."Date" BETWEEN
  ADD_DAYS(CURRENT_DATE,-(wk*2*7))
  AND ADD_DAYS(CURRENT_DATE,-(wk*7))
AND i."ItemCode" LIKE :v_clone||'%'
```

WHAT CHANGED Per-family hardcoded ranges become one dynamic window driven by a config table (product × month → weeks), and the whole run is parameterized by a single clone input — new clones and durations need no code change.

02 What stayed identical, what changed

The migration was a re-engineering of the *how*, never the *what*: business results are unchanged.

PRESERVED 1:1 (BEHAVIOR)

- The staged requirement output — minimum & differentiation units per stage
- The identity **factor = quality × rate**
- The backward stage chain and its safety-stock percentages
- Programmed vs. projected production by week, month and year

RE-ENGINEERED (IMPLEMENTATION)

- Dialect: T-SQL → SQLScript, natively
- 14 → 6 temp tables; 9 → 1 fallback expression
- Null-safe arithmetic on every division
- Data-driven durations & per-clone parameterization
- Explicit, strict typing throughout

In short: the same production numbers are produced by a query that is smaller, safe against runtime failures, strictly typed, and configurable from data rather than code — demonstrated here through faithful transformation patterns, with the client's specific data kept private.